DATA CATALOG & README

JobsDataWarehouse — Job Search Platform

SQL Server | Medallion Architecture | Gold Layer

Version 1.0 | 30 April 2026

1. Project Overview

This document is the official data catalog and technical README for the JobsDataWarehouse project. It describes the architecture, table definitions, column-level metadata, and data lineage for all objects in the Gold layer of the job search platform data warehouse.

Architecture pattern:

```
Bronze  →  Silver (jobs_staging)  →  Gold (star schema)
```

Database:

```
localhost\SQLEXPRESS  →  JobsDataWarehouse
```

Schema:

```
gold.*   (dim_date, dim_job_type, dim_location, fact_jobs)
```

2. Table Inventory

All tables in the JobsDataWarehouse across the Silver and Gold layers:

Table Name	Schema	Type	Description
dim_date	gold	Dimension	Date attributes for time-based analysis
dim_job_type	gold	Dimension	Job type classification (full-time, part-time, etc.)
dim_location	gold	Dimension	Location attributes including remote flag
fact_jobs	gold	Fact	Central fact table — job listings with all FK references
jobs_staging	silver	Staging	Cleaned and standardized raw job data from sources

3. Table Definitions

3.1 gold.dim_date

Stores all date-related attributes used for time-based slicing and aggregation. Populated from the posting dates in silver.jobs_staging.

Column	Data Type	Key	Description
date_sk	INT	PK	Surrogate key (auto-increment)
full_date	DATE		Full calendar date (YYYY-MM-DD)
day	INT		Day of month (1–31)
month	INT		Month number (1–12)
year	INT		Calendar year
quarter	INT		Quarter (1–4)
day_of_week	VARCHAR(20)		Day name (Monday, Tuesday, etc.)

3.2 gold.dim_job_type

Lookup table for job classification types. Contains one row per unique job type extracted from the staging layer.

Column	Data Type	Key	Description
job_type_sk	INT	PK	Surrogate key (auto-increment)
job_type_name	VARCHAR(50)		Job type label (Full-time, Part-time, Contract, Internship)

3.3 gold.dim_location

Stores unique location values and their remote/on-site classification. Used to enable geographic analysis of job postings.

Column	Data Type	Key	Description
location_sk	INT	PK	Surrogate key (auto-increment)
location_name	VARCHAR(100)		City, region or country name
is_remote	VARCHAR(10)		Remote flag (Yes / No / Hybrid)

3.4 gold.fact_jobs

The central fact table of the star schema. Each row represents one job posting. Foreign keys link to the three dimension tables. Measures include job metadata rather than numeric aggregates.

Column	Data Type	Key	Description
job_sk	BIGINT	PK	Surrogate key (auto-increment)

Column	Data Type	Key	Description
date_sk	INT	FK	References gold.dim_date(date_sk)
location_sk	INT	FK	References gold.dim_location(location_sk)
job_type_sk	INT	FK	References gold.dim_job_type(job_type_sk)
source	VARCHAR(50)		Data source (LinkedIn, Indeed, etc.)
business_id	VARCHAR(100)		Unique business/company identifier from source
title	VARCHAR(255)		Job title as posted
company	VARCHAR(255)		Company name
is_remote	VARCHAR(10)		Remote flag on the job posting
description	VARCHAR(MAX)		Full job description text
job_url	VARCHAR(500)		Direct URL to job listing

4. Relationships & Star Schema

The Gold layer implements a star schema with fact_jobs at the center:

```
dim_date      (date_sk PK)      ──<  fact_jobs  (date_sk FK)
dim_location  (location_sk PK)  ──<  fact_jobs  (location_sk FK)
dim_job_type  (job_type_sk PK)  ──<  fact_jobs  (job_type_sk FK)
```

All foreign keys in fact_jobs are nullable (NULL) to accommodate incomplete staging records without breaking the load pipeline.

5. Data Lineage

End-to-end flow from raw sources to the Gold layer:

Layer	Object	Source	Transformation
Bronze	raw_jobs	Web scraping / APIs / CSV	No transformation — raw ingestion
Silver	jobs_staging	Bronze raw_jobs	Clean nulls, standardize types, deduplicate
Gold	dim_date	silver.jobs_staging	Extract and normalize date fields
Gold	dim_job_type	silver.jobs_staging	Distinct job type values
Gold	dim_location	silver.jobs_staging	Distinct location + remote flag
Gold	fact_jobs	silver.jobs_staging + all dims	Join dims, resolve FKs, load measures

6. DDL Run Order

Tables must be created in this exact order to satisfy foreign key dependencies:

```
Step 1 — CREATE gold.dim_date
Step 2 — CREATE gold.dim_job_type
Step 3 — CREATE gold.dim_location
Step 4 — CREATE gold.fact_jobs   (references all three dims above)
```

Each DDL block uses IF NOT EXISTS guards so the script is safely re-runnable without dropping existing data.

7. Connection Details

Server:

```
localhost\SQLEXPRESS   (SQL Server 17.0)
```

SSMS connection string:

```
Server Name:      localhost\SQLEXPRESS
Authentication:   Windows Authentication
Database:         JobsDataWarehouse
```

Python (pyodbc) connection string:

```
DRIVER={ODBC Driver 17 for SQL Server};
SERVER=localhost\SQLEXPRESS;
DATABASE=JobsDataWarehouse;
Trusted_Connection=yes;
```

8. Notes & Conventions

Surrogate keys: All dimension tables use IDENTITY(1,1) surrogate keys suffixed with _sk.

Nullable FKs: Foreign keys in fact_jobs are nullable to allow partial loads without pipeline failure.

Re-runnable DDL: All CREATE TABLE statements are wrapped in IF NOT EXISTS checks — safe to execute multiple times.

Schema prerequisite: Run CREATE SCHEMA gold before the DDL if the schema does not yet exist.

Source system: silver.jobs_staging is the single source of truth for all Gold layer loads.